PREGRADO

UNIDAD 1 | OVERVIEW & DESARROLLO DE APIS ROBUSTAS CON FRAMEWORKS DE BACKEND

WEB SERVICES-SECURITY

ASI0705 | Arquitectura de Aplicaciones Web

Logro de la unidad: Al finalizar esta unidad, el estudiante será capaz de desarrollar APIs robustas utilizando el framework Spring Boot, implementando seguridad, integración con bases de datos y , y entendiendo el impacto de las APIs en los negocios.

Data Transfer Object

DTO

DTO significa objeto de transferencia de datos, lo que permite definir un contenedor que contiene un grupo de valores o campos, no métodos que explican cómo se deben pasar los datos entre las capas.

DTO está destinado a operaciones y transferencia de datos, sin embargo, los modelos están destinados a la persistencia de datos.

¿Cuándo usar DTO?

Permite representar datos y cómo se transfieren a las capas de la aplicación.

```java
package pe.edu.upc.BJuanEFSanIsidro.entities;

import ...
@Entity
@Table(name = "Evento")
public class Evento {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int idevento;

    @Column(name = "nombre",length = 35, nullable = false)  3 usages
    private String nombre;

    @Column(name = "descripcion",length = 35, nullable = false)  3 usages
    private String descripcion;

    @Column(name = "fecha", nullable = false)  3 usages
    private LocalDate fecha;

    @Column(name = "capacidadmaxima",length = 35, nullable = false)  3 usa
    private int capacidadmaxima;

    @Column(name = "tipoevento",length = 35, nullable = false)  3 usages
    private String tipoevento;
```

Entity: Evento.java

Tiene varios atributos

DTO: EventoDTO.java
Sólo intercambia 3 atributos con el Front del total declarados en el entity.
Se usan los DTOS que se necesiten.

```
package pe.edu.upc.BJuanEFSanIsidro.dtos;

import java.time.LocalDate;

public class EventoDTO {  no usages

    private int idevento;  2 usages
    private String nombre;  2 usages
    private String descripcion;  2 usages
```

```java
package pe.edu.upc.BKenyiEFSanIsidro.controllers;

import ...

@RestController
@RequestMapping("/eventoshuicho")
public class EventoController {
    @Autowired
    private IEventoService service;

    @GetMapping
    public List<EventoDTO> listar() {
        return service.list().stream().map(x -> {
            ModelMapper m = new ModelMapper();
            return m.map(x, EventoDTO.class);
        }).collect(Collectors.toList());
    }
    @PostMapping
    public void registrar(@RequestBody EventoDTO dto) {
        ModelMapper m = new ModelMapper();
        Evento p = m.map(dto, Evento.class);
        service.insert(p);
    }
```

Interacción del controller Con el DTO

https://www.jwt.io/

JSON Web Token

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define una manera compacta y autocontenida para transmitir de forma segura información entre parties a manera de un JSON object.

Esta información es confiable y verificable dado que está firmada digitalmente.

JWTs pueden firmarse usando un secret (con el algoritmo HMAC) o un public/private key pair usando RSA ó ECDSA.

Cuándo usar JSON Web Tokens

Escenarios de uso:

Authorization. El escenario más común. Una vez que el usuario inicia sesión, cada request posterior incluirá el JWT, permitiendo al usuario acceder a routes, services y resources que se estén permitidos para ese token.

Information Exchange. JSON Web Tokens son una manera de transmitir información segura entre parties. Dado que los JWTs se pueden firmar se puede estar seguro de que los senders son quienes dicen ser. Adicionalmente, dado que la firma se calcula usando le header y payload, puede verificarse que el contenido no ha sido manipulado.

Structure

En su forma compacta, JSON Web Tokens consisten de tres partes separadas con puntos (.):

Header

Payload

Signature

xxxxx.yyyyy.zzzzz

Header

El header normalmente consta de dos partes: tipo de token, que es JWT y el signing algorithm que se usa, como HMAC SHA256 o RSA.

Este JSON tiene un encoding usando Base64Url para la primera parte del JST.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

Contiene los claims. Claims son los statements acerca de un entity (normalmente user) y datos adicionales.

Hay tres tipos de claims: registered, public y private claims.

Registered claims. Claims predefinidos no obligatorios pero recomendados, para proporcionar un conjunto de claims que sea útil e interoperable. Entre ellos están iss (issuer), exp (expiration time), sub (subject), aud (audience), entre otros.

Public claims. Definidos a voluntad por los que usen JWTs. Para evitar colisiones deberían estar definidos en un IANA JSON Web Token Registry o estar definidos como URI que contenga un collision resistant namespace.

Private claims. Claims personalizados creados para compartir información entre parties que acuerdan usarlos y no son ni registered ni public claims.

Payload

Por ejemplo

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

Este payload luego es encoded con Base64Url para formar la segunda parte del JSON Web Token.

Signature

Para crear el signature se toma el header, encoded payload, un secret, el algorithm especificado en el header y se firma.
Por ejemplo, usando el HMAC SHA256 algirithm se crea un signature de esta manera.

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret)
```

El signature permite verificar que el mensaje no ha sido alterado en el camino. En caso de signed tokens con private key, permite verificar que el sender del JWT es quien dice ser.

All together

Un JWT con un header y payload que están encoded y signed con


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaXNTb2NpYWwiOnRydWV9.
4pcPyMD09olPSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```


Para comprobar los conceptos

https://jwt.io/#debugger-io

JSON Web Tokens in action

Cuando el usuario inicia sesión con éxito usando sus credenciales, se retorna un JSON Web Token.

Cuando el usuario desea acceder a routes o resources protegidos, el user agent debería enviar el JWT, normalmente en el **Authorization** header usando el esquema **Bearer**.

```
Authorization: Bearer <token>
```

Flujo JWT en Spring

```
Login Request
    ↓
Spring Security
    ↓
Generar JWT
    ↓
Cliente recibe token
    ↓
Cliente envía token en cada request
    ↓
Filtro JWT valida token
    ↓
Acceso al endpoint
```

Dependencias (Maven)

XML

```xml
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-api</artifactId>
 <version>0.11.5</version>
</dependency>

<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-impl</artifactId>
 <scope>runtime</scope>
</dependency>

<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-jackson</artifactId>
 <scope>runtime</scope>
</dependency>
```

AGENDA

JWT

SPRING FRAMEWORK & SECURITY

Spring Security

Spring Initializr permite adicionar Spring Security.

Para adicionar Spring Security Support a una aplicación, se adiciona las dependencias en pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Spring Security

- A partir de Spring Security 6, la configuración basada en WebSecurityConfigurerAdapter fue eliminada.
- Actualmente, la seguridad se define mediante beans de tipo SecurityFilterChain.
- La configuración se realiza utilizando un enfoque funcional con expresiones lambda, lo que mejora la claridad y mantenibilidad del código.
- El método authorizeHttpRequests permite definir qué rutas requieren autenticación y cuáles son públicas.
- Es posible configurar políticas sin estado (STATELESS) para aplicaciones REST.
- Los filtros personalizados se integran mediante addFilterBefore, manteniendo el control sobre el flujo de autenticación.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
        )

        .exceptionHandling(ex -> ex
            .authenticationEntryPoint(authenticationEntryPoint)
        )

        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        );

    http.addFilterBefore(requestFilter,
        UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

Encode JSON Web Token

Obtener un JwtBuilder, establecer los claims y generar el token usando compact() con una clave segura.

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

import java.security.Key;
import java.util.Date;

Key key = Keys.secretKeyFor(SignatureAlgorithm.HS256);

String jwt = Jwts.builder()
        .setSubject("1234567890")
        .setId("d11c845e-8e49-4519-a884-4f3153db6060")
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + 3600000)) // 1 hora
        .claim("name", "John Doe")
        .claim("admin", true)
        .signWith(key)
        .compact();
```

Decode JSON Web Token

Obtener un JwtParserBuilder, configurar la clave de firma y realizar el parsing del token con parseClaimsJws() para validar y extraer los claims.

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.security.Keys;

import java.security.Key;
import java.nio.charset.StandardCharsets;

String secret = "my-super-secret-key-that-is-at-least-256-bits!!";
Key key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

String jws = "eyJhbGciOiJIUzI1NiJ9..."; // JWT de ejemplo

Claims claims = Jwts.parserBuilder()
        .setSigningKey(key)
        .build()
        .parseClaimsJws(jws)
        .getBody();

// Acceso a claims
String subject = claims.getSubject();
String name = claims.get("name", String.class);
Boolean admin = claims.get("admin", Boolean.class);
```

Pruebas Unitarias

La anotación @Slf4j en Spring Boot es una anotación de Lombok que permite integrar SLF4J (Simple Logging Facade for Java) en una clase sin necesidad de declarar manualmente un objeto Logger.

¿Para qué se usa @Slf4j?

- Facilita el uso de logs sin necesidad de crear una instancia de Logger manualmente.
- Permite imprimir mensajes de depuración e información en consola o en un archivo de log.
- Optimiza el código al evitar la declaración explícita de LoggerFactory.getLogger().

Ejemplo de prueba de un servicio con Mockito

```java
@ExtendWith(MockitoExtension.class)
class ProductoServiceTest {

    @Mock
    private ProductoRepository productoRepository;

    @InjectMocks
    private ProductoService productoService;

    @Test
    void obtenerProductos_DeberiaRetornarListaDeProductos() {
        // Given
        Producto p1 = new Producto(1L, "Laptop", 1500.99);
        Producto p2 = new Producto(2L, "Mouse", 25.50);
        when(productoRepository.findAll()).thenReturn(List.of(p1, p2));

        // When
        List<Producto> productos = productoService.obtenerProductos();

        // Then
        assertThat(productos).hasSize(2);
        verify(productoRepository, times(1)).findAll();
    }
}
```

Explicación del código

@Mock simula la base de datos con ProductoRepository.
@InjectMocks inyecta el mock en ProductoService.
when().thenReturn() simula respuestas del repositorio.
verify() verifica que el método findAll() se ejecutó exactamente una vez.

Resultado esperado: La prueba verifica que el servicio retorna la lista correcta de productos.

REFERENCIAS

Para profundizar

https://jwt.io/

https://www.w3.org/wiki/CORS_Enabled

https://spring.io/projects/spring-security

https://www.jsonwebtoken.io/

PREGRADO

Ingeniería de Sistemas de Información
Escuela de Ingeniería de Sistemas y Computación | Facultad de Ingeniería

UPC
Universidad Peruana
de Ciencias Aplicadas

Prolongación Primavera 2390,
Monterrico, Santiago de Surco
Lima 33 - Perú
T 511 313 3333
https://www.upc.edu.pe

exígete, innova

LAUREATE
INTERNATIONAL
UNIVERSITIES